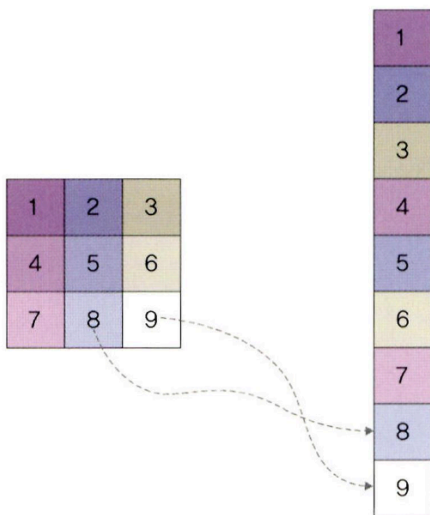


[딥러닝 파이토치 교과서] 5장

5.1 합성곱 신경망

- 정리
 - 딥러닝의 계산 과정은 복잡하고 많은 자원을 요구함
 - 이 문제를 해결하기 위해 합성곱 신경망이 등장
 - 이미지의 국소적 부분을 계산함으로써 시간과 자원을 절약해 이미지의 세밀한 부분까지 분석할 수 있는 신경망

5.1.1 합성곱층의 필요성

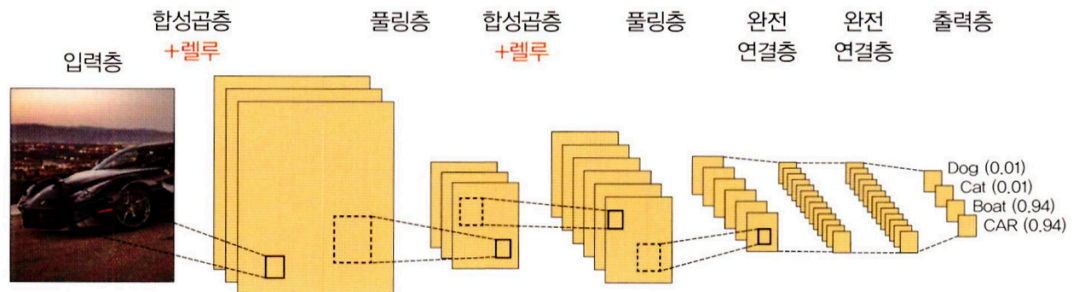


- 기존 이미지 분석
 - 3×3 배열을 오른쪽과 같이 펼쳐 픽셀 별로 가중치를 곱해 은닉층으로 전달
 - 데이터의 공간적 구조를 무시하게 되는 문제 발생
→ 합성곱층의 도입

5.1.2 합성곱 신경망 구조

- 합성곱 신경망(CNN)
 - 음성 인식이나 이미지/영상 인식에서 주로 사용되는 신경망
 - 컬러 이미지 같은 다차원 배열 처리에 특화됨
 - 다음의 다섯 계층으로 구성됨
 - 입력층
 - 합성곱층

- 풀링층
- 완전연결층
- 출력층



○ 대략적인 구조

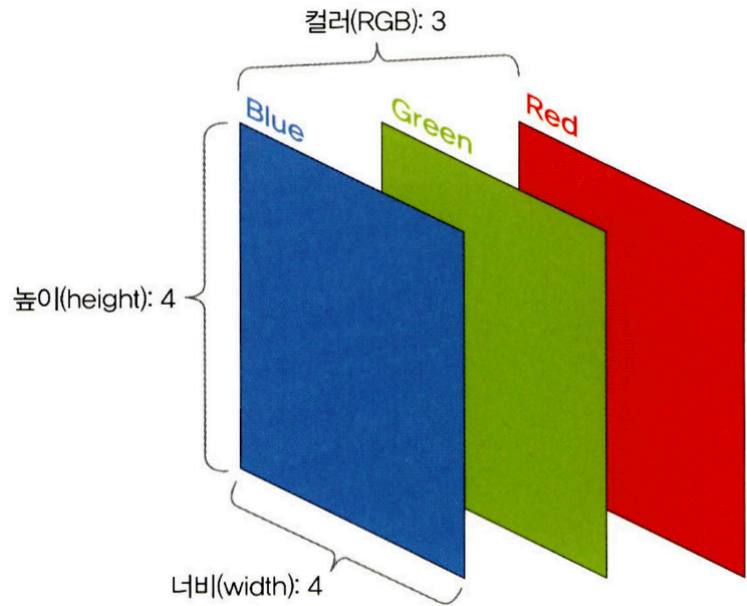
합성곱층과 풀링층을 거치며 입력 이미지의 주요 특성 벡터 추출

→ 주요 특성 벡터들이 완전 연결층을 거치며 1차원 벡터로 변환

→ 출력층에서 활성화 함수인 소프트맥스 함수를 이용해 최종 결과 출력

입력층

- 입력 이미지 데이터가 최초로 거치게 되는 계층
- 이때 이미지는 높이, 너비, 채널의 값을 갖는 3차원 데이터
- 채널은 이미지가 그레이스케일(gray scale)이면 1의 값을, 컬러(RGB)이면 3의 값을 가짐



높이 4, 너비 4, 채널은 RGB를 가짐 → 이미지 형태(shape): (4,4,3)

합성곱층

- 입력 데이터에서 특성을 추출하는 역할 수행
- 특성 추출
 - 입력 이미지를 커널이나 필터를 사용해 특성 감지
 - 커널/필터는 이미지의 모든 영역을 훑으며 특성 추출 → 그 결과물로 특성 맵 (feature map) 산출
 - 커널은 3x3, 5x5 크기로 적용되는 것이 일반적, 지정된 간격(스트라이드, stirde)에 따라 순차적으로 이동

- 스트라이드 1일 때의 이동 과정(ver. 그레이스케일)

이미지 크기(6, 6, 1)

3x3 크기의 필터/커널이 스트라이드 1 간격으로 이동하며 합성곱 연산 수행

- 1단계. 입력 이미지에 3x3 필터 적용

$$(1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$$



입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱한 후 모두 더함

■ 2단계. 필터가 1만큼 이동

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 1$$

▼ 그림 5-5 입력 이미지에 필터가 1만큼 이동



■ 3단계. 필터가 1만큼 두 번째 이동

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 1) = 1$$

▼ 그림 5-6 입력 이미지에 필터가 1만큼 두 번째 이동



■ 4단계. 필터가 1만큼 세 번째 이동

$$(0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) = 3$$

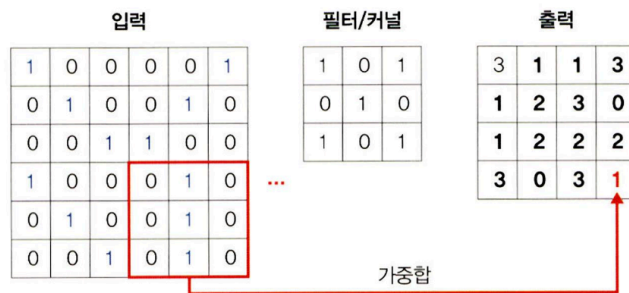
♥ 그림 5-7 입력 이미지에 필터가 1만큼 세 번째 이동



- (반복)
- 17단계. 필터가 1만큼 마지막으로 이동

$$(0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) = 1$$

♥ 그림 5-9 입력 이미지에 필터가 1만큼 마지막으로 이동



⇒ 커널이 스트라이드 간격만큼 순회하며 모든 입력 값과의 합성곱 연산으로 새로운 특성 맵 만들

⇒ 결과적으로 (6, 6, 1) → (4, 4, 1) 크기의 특성 맵으로 줄어듦

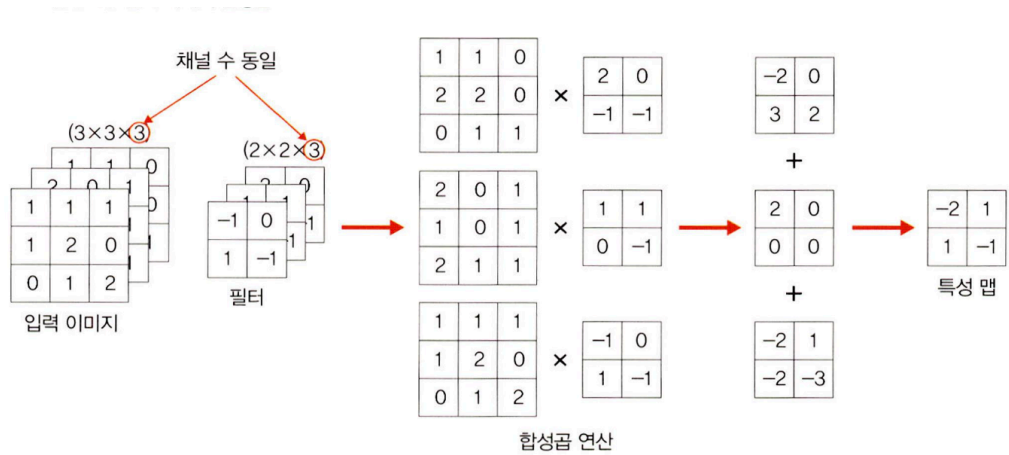
○ 스트라이드 1일 때의 이동 과정(ver. 컬러 이미지)

- 그레이스케일과 구분되는 특징

1. 필터 채널이 3임

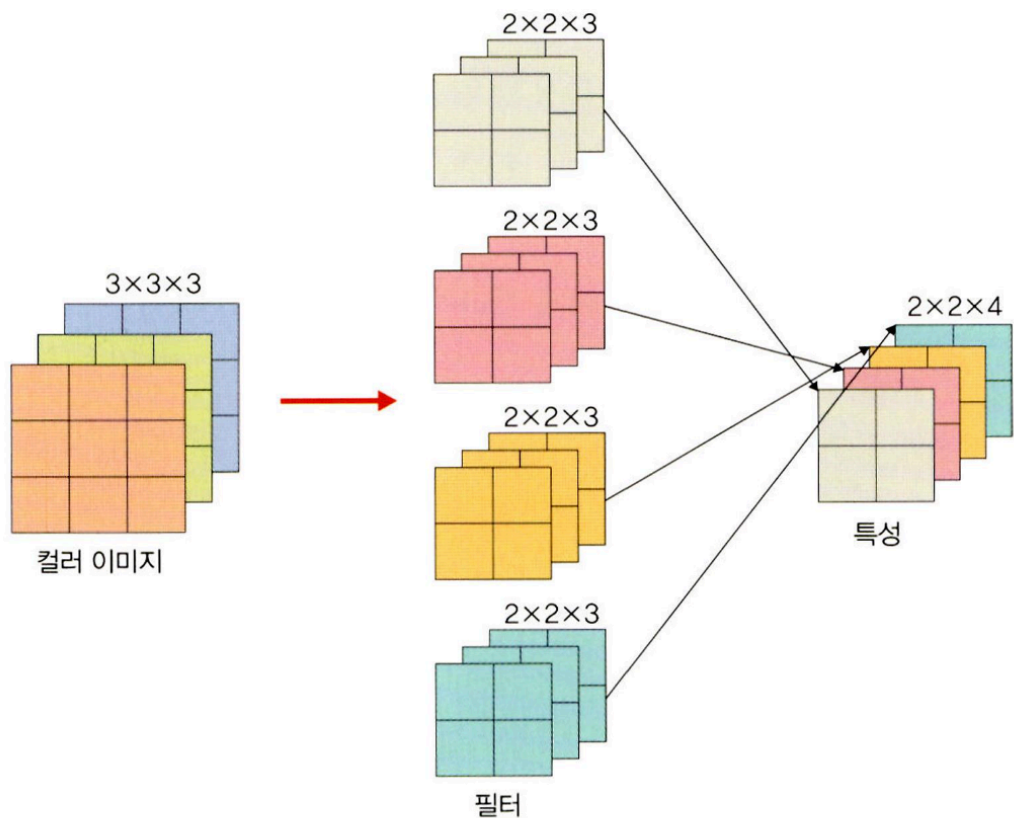
*필터 채널이 3이라고 해서 필터의 개수가 3개인 것이 아님. 필터의 개수는 1개라는 점 주의!

2. RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후, 결과를 더함



각각 서로 다른 가중치로 합성곱을 적용한 후 결과를 더함

◦ 필터가 두 개 이상인 합성곱



필터 각각이 특성 추출 결과의 채널이 되어 앞서 진행했던 것과 동일한 방법으로 계산

◦ 요약 정리

- 입력 데이터: $W_1 \times H_1 \times D_1$ (W_1 : 가로, $\times H_1$: 세로, $\times D_1$: 채널 또는 깊이)
- 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P
- 출력 데이터
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$
 - $D_2 = K$

풀링층

- 특성 맵의 차원을 다운 샘플링하여 연산량 감소시킴
- 주요한 특성 벡터를 추출해 효과적인 학습을 도움
- 풀링 연산
 1. 최대 풀링(max pooling): 대상 영역에서 최댓값 추출
 2. 평균 풀링(average pooling): 대상 영역에서 평균 반환

→ 대부분의 합성곱 신경망에서는 최대 풀링이 사용됨

평균 풀링이 각 커널 값을 평균화시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있기 때문

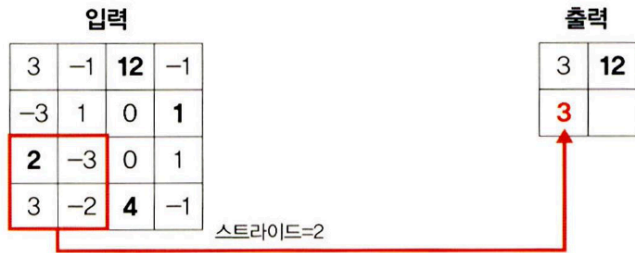
■ 최대 풀링의 연산 과정



3, -1, -3, 1 값 중에서 최댓값 3 선택



12, -1, 0, 1 값 중에서 최댓값 12 선택



-2, -3, 3, -2 값 중에서 최댓값 3 선택



0, 1, 4, -1 값 중에서 최댓값 4 선택

■ 평균 풀링의 연산 과정

$$0 = (3 + (-1) + (-3) + 1) / 4$$

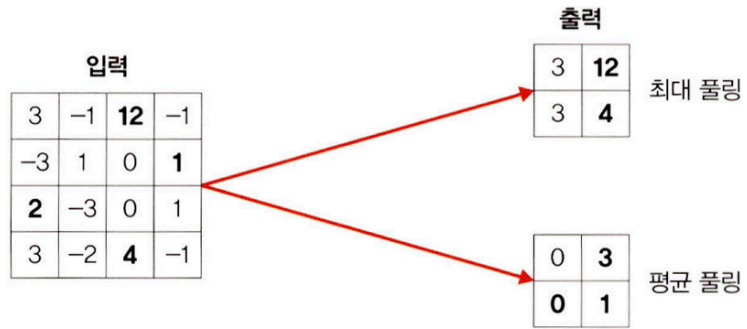
$$3 = (12 + (-1) + 0 + 1) / 4$$

$$0 = (2 + (-3) + 3 + (-2)) / 4$$

$$1 = (0 + 1 + 4 + (-1)) / 4$$

최대 풀링과 유사한 방식으로 진행하되, 각 필터의 평균으로 계산

cf) 최대 풀링과 평균 풀링의 결과 비교

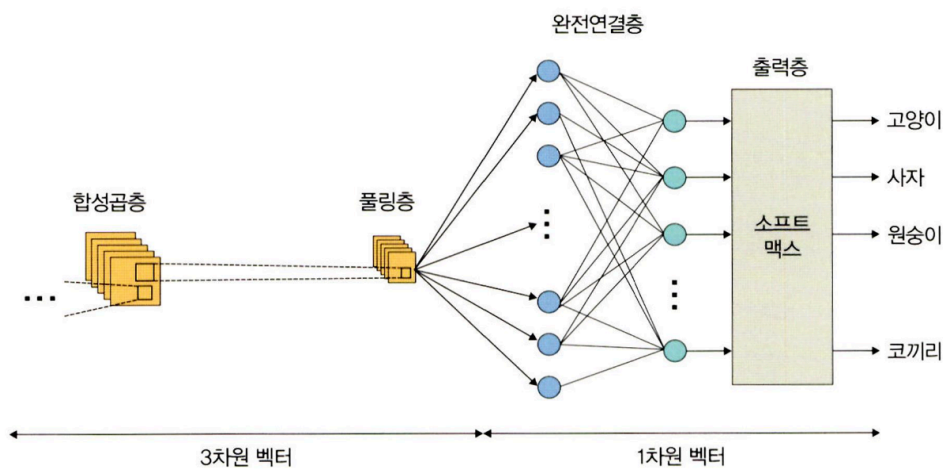


■ 요약 정리

- 입력 데이터: $W_1 \times H_1 \times D_1$
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

최대/평균 풀링의 파라미터는 동일

완전연결층



차원 축소된 특성 맵은 최종적으로 완전연결층으로 전달되어 3차원 벡터에서 1차원 벡터로 펼쳐짐

출력층

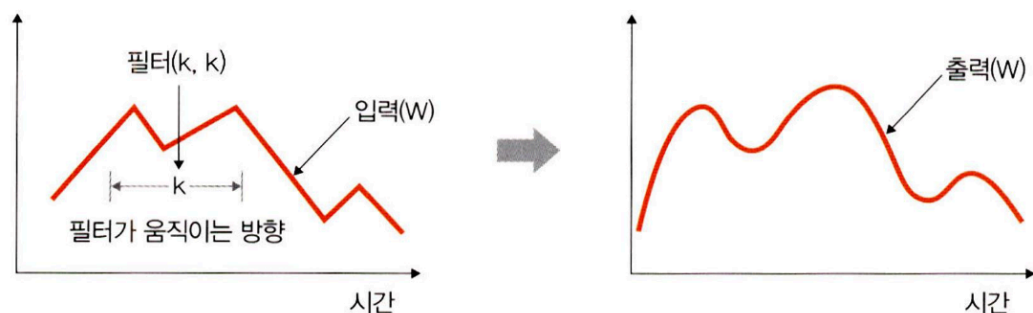
- 소프트맥스 활성화 함수 사용해 이미지가 각 레이블에 속할 확률 값 출력(0~1 사이)
- 그 중 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정

5.1.3 1D, 2D, 3D 합성곱

이동하는 방향 수와 출력 형태에 따라 1D, 2D, 3D로 분류

1D 합성곱

- 필터가 시간을 축으로 좌우(한 개의 방향)로만 이동할 수 있는 합성곱
- 입력(W)과 필터(k)에 대한 출력 W
ex. 입력 [1, 1, 1, 1, 1] / 필터 [0.25, 0.5, 0.25]
출력 [1, 1, 1]
- 1D의 출력 형태
- 그래프 곡선 완화를 위해 주로 사용

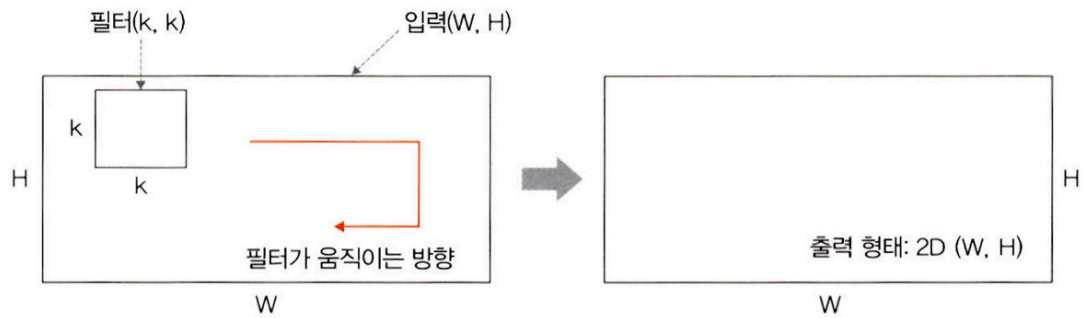


- 입력: W 너비(Width)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width)

2D 합성곱

- 방향 두 개로 움직이는 합성곱
- 입력(W, H)과 필터(k, k)에 대한 출력 (W, H)

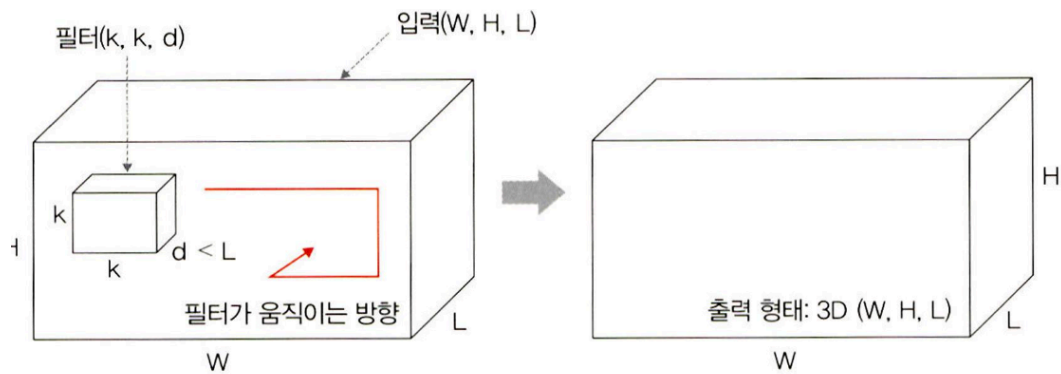
- 2D 형태의 출력 형태



- 입력: W 너비(Width), H 높이(Height)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width), H 높이(Height)

3D 합성곱

- 방향 세 개로 움직이는 합성곱
- 입력(W, H, L)과 필터(k, k, d)에 대한 출력 (W, H, L)
- 3D 형태의 출력
- $d < L$ 을 유지하는 것이 중요



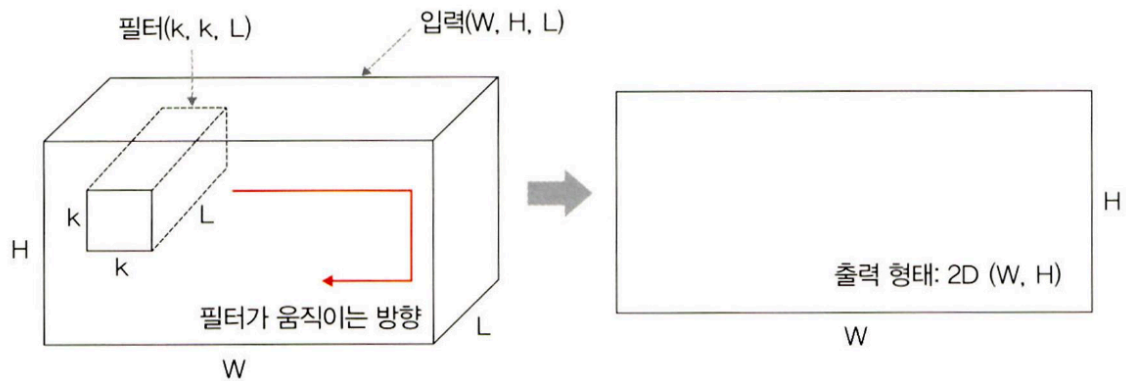
- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k \times d$ (높이 $\times$ 너비), d: 깊이(depth)
- 출력: W 너비(Width), H 높이(Height), L 길이(Length)

3D 입력을 갖는 2D 합성곱

- 입력이 3D 형태임에도 2D 행렬의 출력 형태를 취하는 합성곱

필터에 대한 길이가 입력 채널의 길이와 같아야 하기 때문에 이와 같은 형태가 만들어짐

- 입력(W, H, L)과 필터(k, k, L)에 대한 출력(W, H)
- 2D 행렬의 출력 형태
- 대표적인 사례로 LeNet-5, VGG가 있음(추후 6장에서 다룸)



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

필터는 그림과 같이 두 방향으로 움직임

1x1 합성곱

- 3D 형태의 입력
- 입력(W, H, L)과 필터($1, 1, L$)에 대한 출력(W, H)

5.2. 합성곱 신경망 맛보기

[Week3_예습과제_박린.ipynb](#)